Project Report: CVM++

Architecture and Implementation of a Custom Scripting Language and Virtual Machine

1. Executive Summary

CVM++ is a lightweight, Turing-complete custom scripting language built entirely from scratch in C++. The objective of this project was to demystify the pipeline of high-level code execution by implementing a full Ahead-of-Time (AOT) compiler and runtime environment. The system successfully translates human-readable scripts into a custom Abstract Syntax Tree (AST), compiles that tree into raw binary machine code, and executes the bytecode using a custom stack-based Virtual Machine.

2. System Architecture

The CVM++ engine is divided into two distinct execution phases, mirroring professional compiled languages like Java or C#:

1. **The Compilation Phase:** Reads `.cvm` text files and generates `.bin` bytecode files.
2. **The Runtime Phase:** Reads `.bin` bytecode files and executes them in memory.

This architecture is powered by four primary modules:

2.1 Lexical Analysis (The Lexer)

The Lexer (`Lx`) is responsible for reading the raw source code string and converting it into a sequence of manageable Tokens (`Tk`).

- It systematically ignores whitespace while grouping characters into logical units (e.g., `TkTp::NUM` for integers, `TkTp::ID` for variables).
- It distinguishes between standard mathematical operators (`+`, `-`, `*`, `/`) and language-specific keywords (`let`, `if`, `while`, `input`, `print`).
- **Design Note:** The lexer was explicitly designed to support professional variable naming conventions, utilizing `std::isalnum` alongside explicit checks to allow underscores (_) in variable names.

2.2 Syntax Analysis (The Parser)

The Parser consumes the token stream and constructs an Abstract Syntax Tree (AST). It utilizes **Recursive Descent Parsing** to establish strict operator precedence.

- **Mathematical Precedence (PEMDAS):** The parser hierarchy guarantees that multiplication and division (`factor()`) are evaluated before addition and subtraction (`term()`).

- **Control Flow Blocks:** The parser handles nested logic by grouping statements between `{` and `}` into `BlockStmt` nodes, allowing `if` branches and `while` loops to execute multiple lines of code sequentially.

2.3 Ahead-of-Time Bytecode Compiler

The Compiler traverses the AST and flattens the hierarchical nodes into a linear array of 8-bit integers (`uint8_t`), representing custom Operation Codes (Opcodes).

- **Memory Optimization:** Instead of saving variable names as string metadata, the compiler uses a Symbol Table to dynamically map string IDs to 8-bit memory addresses, significantly speeding up runtime execution.
- **Control Flow Jumping:** To implement `if` statements and `while` loops, the compiler calculates the exact byte-length of code blocks and injects 16-bit address offsets into the bytecode (`JMP` and `JMP_FALSE`). This allows the Virtual Machine to skip over code or loop backward in memory.
- **Disassembler / Debug Matrix:** The compiler includes a built-in disassembler (`--debug` flag) that translates the raw hexadecimal machine code back into human-readable instructions prior to binary serialization, providing a vital tool for debugging stack operations.

2.4 The Stack-Based Virtual Machine

The final runtime engine executes the generated bytecode array. CVM++ utilizes a stack-based architecture rather than a register-based one.

- **Execution Model:** The VM iterates over the bytecode using an Instruction Pointer (`ip`). Mathematical operations pull their operands directly off the top of the stack (`pop()`) and push the calculated result back on (`push()`).
- **Variable Storage:** Variables are stored in a dedicated `globals` array, accessed via `SET_VAR` and `GET_VAR` opcodes.
- **Interactive I/O:** The VM supports fully interactive terminal input through the custom `READ` opcode, which pauses the Instruction Pointer, triggers C++ `std::cin` to accept user input, and pushes the user's integer directly to the top of the stack for immediate evaluation.

3. Supported Language Features

The resulting language is fully Turing-complete and supports the following features:

- **Dynamic Variable Binding:** Creation (`let x = 5`) and reassignment (`x = x + 1`).
- **Conditionals:** Standard `if / else` branching block execution.
- **Loops:** Infinite and conditional `while` loops.
- **I/O:** User input via `input` and console output via `print`.

4. Conclusion

Building CVM++ successfully demonstrated the complex lifecycle of code execution. One of the most significant challenges was managing stack integrity—specifically, ensuring the compiler generated the exact mathematical expressions before emitting a `SET_VAR` opcode to prevent fatal "Stack Underflow" runtime errors. The final product successfully proves that a robust, interactive programming language can be engineered from fundamental C++ principles without relying on external parsing libraries.